

Relatório PBL - Maratona de Programação

Gabriel Oliveira de Freitas

Departamento de Exatas (DEXA) – Universidade Estadual de Feira de Santana (UEFS), Av. Transnordestina, s/n - Novo Horizonte - CEP 44036-900 Feira de Santana - Bahia - Brasil

goliveira0403@gmail.com

Resumo. *A Universidade Estadual de Feira de Santana deseja organizar uma maratona de programação para os alunos de três cursos: Engenharia de Computação, Sistemas de Informação e Ciências da Computação. O evento consistirá na resolução de problemas, por cinco equipes, em um determinado tempo. A equipe que acumular mais pontos será a vencedora da maratona. Posto isso, os alunos de Engenharia de Computação devem desenvolver um sistema em Python, que será utilizado para o gerenciamento da maratona. O sistema registrará as pontuações, os problemas resolvidos e o tempo gasto pelas equipes. Além disso, o sistema gerará o ranking e exibirá outras informações relevantes sobre o evento. O código foi concluído com êxito, atendendo a todas as exigências.*

1. Introdução

Uma maratona será organizada pela Universidade Estadual de Feira de Santana (UEFS), que está prevista para acontecer na data: 20/10/2024. A maratona terá a participação dos cursos de Engenharia de Computação, Sistemas de Informação e Ciências da Computação, que serão divididos em 5 equipes, compostas por 2 ou 3 alunos, que terão o objetivo de solucionar a maior quantidade de problemas de programação possíveis em um determinado tempo. Os problemas são divididos em três dificuldades: fácil, médio e difícil, cada uma dessas dificuldades terá um valor diferente, que será definido antes da competição começar. A equipe campeã será aquela com a maior pontuação ao final da competição. Se houver empate na pontuação final, a equipe que resolveu a maior quantidade de questões difíceis ficará à frente. Se o empate persistir, a equipe que fez em menos tempo vencerá a maratona.

O objetivo deste relatório é apresentar um sistema desenvolvido na linguagem Python para organizar o funcionamento da maratona e gerenciar a pontuação e o tempo das equipes durante o evento. O sistema deve registrar o valor das dificuldades (fácil, médio e difícil), o nome das 5 equipes, que tipo de problema foi resolvido por cada equipe e o tempo necessário para a realização desses problemas. Ele também deve somar a pontuação e o tempo a cada problema resolvido para atualizar o ranking geral da maratona.

Esse relatório está dividido em 4 tópicos: 1) Introdução: Descrição e explicação do problema e do sistema solicitado para sua resolução. 2) Metodologia: Descrição das sessões, requisitos e descrição de alto nível, ordem de codificação e ficha técnica. 3)

Resultados e discussões: Manual de uso, dados de entrada e dados de saída, testes e erros. 4) Conclusão: Descrição das conclusões do que foi feito.

2. Metodologia

Durante as três semanas de sessões da metodologia Problem-Based Learning (PBL), foram discutidas maneiras de solucionar o problema, abordando diversas questões sobre algumas especificidades do sistema a ser desenvolvido. Em conjunto, foi analisado o que deveria ser feito e foi decidido que o peso dos problemas deveria ser definido pelo usuário, sendo que o problema médio não poderia ter um peso maior que o difícil, e o problema fácil não poderia ser maior que o médio. Também foi discutido o ranking, onde foi estabelecido que esse deveria ser atualizado a cada rodada de problemas.

2.1. Sessões Tutoriais

Para o desenvolvimento do algoritmo, ocorreram 3 sessões tutoriais. Durante elas, o grupo compartilhou informações e conhecimento com os integrantes, discutiu possibilidades para o funcionamento do código e definiu os requisitos. A cada sessão, três pessoas assumiram funções diferentes no grupo, sendo elas: coordenador, que assumia um papel de liderança do grupo; secretário de quadro, que escrevia no quadro as ideias, os fatos, as questões e as metas; e o secretário de mesa, que fazia um relatório sobre a sessão.

2.2. Requisitos do Sistema

É requisito que ao final do evento o sistema exiba:

- O ranking ordenado;
- A quantidade de problemas por dificuldade que cada equipe resolveu;
- O nome da equipe vencedora e sua pontuação total;
- O nome da equipe que resolveu mais problemas difíceis;
- A média de pontos por equipe;
- O tempo total gasto pela equipe vencedora.

Além disso, é necessário que o código permita que o usuário escolha os pesos das questões. Dentre esses requisitos, o mais discutido foi o ranking ordenado. Para a resolução desse problema, foram utilizadas diversas soluções, incluindo o uso de condicionais, para determinar a posição de cada equipe, estruturas de repetição como `for` ou `while`, e a troca de variáveis para mudar as colocações das equipes.

2.3. Descrição do Algoritmo

O algoritmo foi desenvolvido utilizando estruturas condicionais (`if`, `elif` e `else`) e estruturas de repetição (`for` e `while`). O código começa atribuindo `"True"` à variável `"continuar"`. Em seguida, são iniciadas estruturas de repetição `"while continuar:"` para fazer as validações das variáveis de peso das dificuldades e do nome das equipes. Enquanto o usuário não digitar o que foi pedido, o código volta a solicitar a entrada. Em seguida, são inicializadas as variáveis para somar a pontuação, o tempo e a quantidade

de problemas que cada equipe resolveu, tanto no total quanto por dificuldade, todas inicializadas com o valor 0, para contar ou somar dentro da estrutura de repetição while que vem a seguir.

Após inicializar as variáveis, começa a parte principal do código. Antes disso, a string '1' é atribuída à variável escolha. Em seguida, inicia-se a parte essencial para o funcionamento do código, onde começa uma estrutura de repetição: “while escolha == '1':”. Dentro dessa estrutura está inserida a maior parte do código, que começa com um menu de escolha, onde o usuário pode escolher entre três opções (1, 2, 3). A primeira opção é registrar a pontuação e o tempo de alguma equipe específica. Nesta opção, a partir de uma estrutura condicional com if e elif, dependendo da equipe escolhida, são somados a pontuação, o tempo e a contagem de questões totais e por dificuldade.

A segunda opção do menu serve para mostrar o ranking, que é ordenado da 1ª à 5ª equipe. Para isso, ocorre uma mudança de nome para as variáveis, com o objetivo de não causar conflitos no código. Como é possível visualizar na figura a seguir:

```
#Se a escolha for '2', mostra o ranking das equipes.
elif escolha == '2':
    #Atribui outras variáveis para o nome da equipe, pontuação, dificuldade e tempo, respectivamente,
    #para serem utilizadas exclusivamente no ranking sem dar conflito no resto do código.
    eq1, pont1, dif1, tempo1 = e1, pe1, contadorD1, te1
    eq2, pont2, dif2, tempo2 = e2, pe2, contadorD2, te2
    eq3, pont3, dif3, tempo3 = e3, pe3, contadorD3, te3
    eq4, pont4, dif4, tempo4 = e4, pe4, contadorD4, te4
    eq5, pont5, dif5, tempo5 = e5, pe5, contadorD5, te5
```

Figura 1. Mudanças de variáveis

O ranking é moldado dentro de uma estrutura de repetição for com um range de 5 repetições. No print do ranking, a ordem vai da primeira equipe nomeada até a última, sem ordem, por isso, dentro do laço de repetição for, ocorrem trocas de variáveis de equipe, pontuação, tempo e questões difíceis resolvidas, com a utilização da estrutura condicional if, portanto, se a equipe 2 tiver uma pontuação maior, ou igual, mas com algum dos critérios de desempate a favor dela que a equipe 1, a variável da equipe 2 passa a ser a variável da equipe 1 e vice-versa, isso acontece sucessivamente com as outras equipes, assim organizando o ranking. Isso está representado na figura a seguir:

```
ordena as equipes com base na pontuação, questões difíceis resolvidas e tempo.
#no print do ranking, a variável tomada como base pra ser a primeira é a do eq1, a segunda é a do eq2 e assim sucessivamente,
#o mesmo acontece com a pontuação, as questões difíceis resolvidas e o tempo gasto, portanto, as trocas desse loop são baseadas nisso.
for ranking in range(5): #loop para ordenar as equipes com base na pontuação, dificuldade e tempo.
    #comparação entre equipe 2 e equipe 1
    if pont2 > pont1 or pont2 == pont1 and dif2 > dif1 or pont2 == pont1 and dif2 == dif1 and tempo2 < tempo1:
        #Troca de posições se a equipe 2 for melhor que a equipe 1. (Troca de variáveis).
        eq1, eq2 = eq2, eq1
        pont1, pont2 = pont2, pont1
        dif1, dif2 = dif2, dif1
        tempo1, tempo2 = tempo2, tempo1
    #comparação entre equipe 3 e equipe 2
    if pont3 > pont2 or pont3 == pont2 and dif3 > dif2 or pont3 == pont2 and dif3 == dif2 and tempo3 < tempo2:
        #Troca de posições se a equipe 3 for melhor que a equipe 2.
        eq2, eq3 = eq3, eq2
        pont2, pont3 = pont3, pont2
        dif2, dif3 = dif3, dif2
        tempo2, tempo3 = tempo3, tempo2
    #comparação entre equipe 4 e equipe 3.
    if pont4 > pont3 or pont4 == pont3 and dif4 > dif3 or pont4 == pont3 and dif4 == dif3 and tempo4 < tempo3:
        #Troca de posições se a equipe 4 for melhor que a equipe 3.
        eq3, eq4 = eq4, eq3
        pont3, pont4 = pont4, pont3
        dif3, dif4 = dif4, dif3
        tempo3, tempo4 = tempo4, tempo3
    #comparação entre equipe 5 e equipe 4.
    if pont5 > pont4 or pont5 == pont4 and dif5 > dif4 or pont5 == pont4 and dif5 == dif4 and tempo5 < tempo4:
        #Troca de posições se a equipe 5 for melhor que a equipe 4.
        eq4, eq5 = eq5, eq4
        pont4, pont5 = pont5, pont4
        dif4, dif5 = dif5, dif4
        tempo4, tempo5 = tempo5, tempo4
```

Figura 2. Loop for com troca de variáveis para ordenação

A terceira opção do menu tem o objetivo de finalizar a maratona, para isso é utilizado o controle de fluxo, break, que para de executar tudo que está dentro do while principal e parte para as informações da maratona. Nas informações aparece tudo que é requisitado, o ranking feito dentro do loop while é repetido para ser mostrado no final, a quantidade de problemas resolvidos pela equipe no total e por dificuldade, a equipe vencedora e o total de pontos feitos por ela, através de uma estrutura de condição com if e elif, o programa mostra a equipe ou as equipes que fizeram a maior quantidade de questões difíceis, em seguida mostra a média de pontos por questão de cada equipe e da competição no geral, utilizando um if e else em caso de divisão por 0 e por fim, é mostrado o tempo gasto pela equipe vencedora.

2.4. Ordem de Codificação

O algoritmo foi desenvolvido em 4 semanas, sendo que em 3 delas houve sessões PBL. Na primeira semana, foi criada a base do programa com as entradas para os pesos das questões e para os nomes das equipes. Além disso, foi criado o menu inicial, apenas com as opções de registrar pontuação e tempo. Dentro do registro, eram somadas as pontuações, o tempo e eram contadas as questões totais e por dificuldade das equipes, e a opção de finalizar o programa. Na segunda semana, foram desenvolvidas as validações das entradas de peso das dificuldades, nomes das equipes e a escolha no menu para impedir que o código apresentasse erros. Na terceira semana, o ranking foi desenvolvido com os critérios de desempate para ordenação das equipes e foram adicionadas as informações finais da maratona que são requisitadas, finalizando o código. A quarta semana foi dedicada a testes de erros, ajustes pequenos e ao desenvolvimento do fluxograma. (Apêndice A)

2.5. Ficha Técnica

O sistema operacional utilizado para o desenvolvimento do programa foi o Windows 11 Home Single Language. A linguagem de programação escolhida foi Python, versão 3.12.4. A IDE (Ambiente de Desenvolvimento Integrado) escolhida foi o Visual Studio Code (VSCode), versão 1.93.0.

3. Resultados e Discussões

3.1. “Manual” de Uso

Ao iniciar o programa, o usuário deverá definir os pesos para os problemas de três dificuldades (fácil, médio, difícil), sendo que o valor da questão fácil deve ser maior que 0 e menor que o valor da questão média, o valor da questão média deve ser maior que 1 e menor que o valor da questão difícil, e o valor da questão difícil deve ser maior que 2, para impedir que o valor da questão fácil seja menor ou igual a 0. Em seguida, o usuário definirá o nome das 5 equipes que participarão da competição. O nome das equipes não pode ser vazio nem igual ao de outra equipe. Após a definição dos nomes das equipes, a maratona começará, onde o usuário terá acesso a um menu com 3 opções, nas quais deverá digitar o número da opção desejada, sendo elas:

(1) Registrar pontuação de uma equipe;

(2) Mostrar ranking;

(3) Finalizar maratona.

Ao escolher a opção 1, será solicitado que o usuário digite o nome da equipe que resolveu o problema. O nome da equipe deve ser escrito corretamente, caso contrário, aparecerá uma mensagem de erro, e será solicitado novamente o nome da equipe. Após a escolha da equipe, o usuário deverá registrar a dificuldade do problema e o tempo em segundos necessário para sua realização. Depois disso, o usuário será redirecionado para o menu, onde poderá escolher novamente uma das três opções.

Ao escolher a opção 2, será exibido o ranking ordenado da primeira à última equipe, com os critérios de desempate, para o usuário. A opção de exibir o ranking pode ser solicitada a qualquer momento da maratona, quantas vezes o usuário desejar.

Ao escolher a opção 3, a maratona será finalizada e serão exibidos o ranking, o campeão da maratona e suas estatísticas, como qual ou quais equipes resolveram mais problemas difíceis, quantos problemas no total e por dificuldade cada equipe resolveu, além da média de pontos por questão de cada equipe e da competição.

3.2. Dados de Entrada e Dados de Saída

Os dados de entrada são as informações digitadas pelo usuário, como os pesos das questões, nomes das equipes, a opção escolhida no menu, e a equipe para registrar pontuação e tempo.

Os dados de saída são as informações que o sistema fornece ao usuário após processar as entradas, exibindo o que foi solicitado. Quando o usuário pede para exibir o ranking no menu, ocorre uma saída de dados, onde o ranking ordenado é mostrado, quando o usuário pede para finalizar a maratona, o ranking final e outras informações sobre a competição são exibidos. Além disso, a saída de dados ocorre nas validações das entradas; se o usuário digitar algo incorreto, uma mensagem de erro aparecerá.

3.3. Testes

Durante as semanas de desenvolvimento, diversos testes foram realizados para garantir o bom funcionamento do código. O primeiro teste foi construir a estrutura de repetição principal dentro de um loop for. Entretanto, percebeu-se que seria melhor utilizar o loop while, uma vez que este oferece mais flexibilidade e não possui a limitação de iterações do for. Vários testes de validação foram feitos utilizando o while para impedir que o sistema apresentasse erros. Um dos testes mais trabalhosos foi o do ranking ordenado com critérios de desempate, pois a implementação apenas com estruturas condicionais exigiu muitas linhas de código.

3.4. Erros

O código final não apresenta erros, pois todas as validações foram feitas. Entretanto, durante o desenvolvimento, alguns erros foram encontrados, como nas validações dos

pesos das dificuldades. Por exemplo, se o usuário digitasse letras, o código gerava um erro. Para resolver isso, utilizou-se a manipulação de string “isdigit()”, para verificar se o que foi digitado era um número. Caso não fosse, era solicitado que o usuário inserisse o peso da dificuldade novamente, evitando a quebra do código.

4. Conclusão

O programa cumpriu o que foi exigido, que era criar uma maratona de programação onde o usuário escolhe o peso para as questões, define o nome das equipes, o ranking é exibido e as informações finais exigidas pelo cliente aparecem. Além disso, o programa apresenta a média de pontuação da competição por questão e um menu com a opção de mostrar o ranking a qualquer momento, de forma ilimitada.

É importante ressaltar que o código tem algumas limitações. Por exemplo, se houver empate em todos os critérios, não haverá o anúncio de mais de um vencedor, apenas um, que será o que foi registrado primeiro. Além disso, há uma limitação estética, já que o programa funciona apenas no terminal.

Ademais, é fundamental destacar a importância da metodologia Problem-Based Learning (PBL), que desempenha um papel crucial no aprendizado de programação do desenvolvedor do código, por meio de uma situação prática que permite o ganho de conhecimento e das habilidades necessárias a um programador.

5. Bibliografia Consultada

PYTHON SOFTWARE FOUNDATION. Python 3.12.6 Documentation. Disponível em: <https://docs.python.org/3/>. Acesso em: 02/09 set. 2024.

AWARI. Python: aprenda a usar a função isdigit para validar números. Disponível em: <https://awari.com.br/python-aprenda-a-usar-a-funcao-isdigit-para-validar-numeros/>. Acesso em: 05/09 set. 2024.

Apêndice A - Fluxograma, PBL 01

<https://drive.google.com/file/d/1BlenIKLBSC9V6ARSHHm5H9vC43ajFDtM/view?usp=sharing>